

Checklist

Platform Engineering Checklist

Platform Foundations

- ☐ Define the platform as an **internal product**, not just a collection of tools.
- ☐ Identify the application teams that will use the platform.
- ☐ Assign a platform team to develop and operate it.
- ☐ Provide self-service APIs, tools, services, knowledge, and support.
- ☐ Establish clear platform boundaries: what the platform supports and what it intentionally does not.
- ☐ Measure whether the platform reduces coordination and operational effort for application teams.

Reduce the “Over-General Swamp”

- ☐ Inventory the OSS, cloud services, infrastructure tools, and bespoke systems currently in use.
- ☐ Identify duplicated or overlapping technologies.
- ☐ Locate application-specific integration “glue.”
- ☐ Identify systems that are expensive to upgrade, migrate, or maintain.
- ☐ Reduce unnecessary technology choices where they create long-term maintenance costs.
- ☐ Replace repeated per-application integrations with shared platform capabilities.
- ☐ Prefer abstractions that hide implementation complexity from application teams.

Treat the Platform as a Product

- ☐ Interview application developers and understand their actual needs.
- ☐ Prioritize user experience rather than infrastructure-team convenience.
- ☐ Create a curated set of supported capabilities instead of exposing every possible option.
- ☐ Deliver improvements incrementally.
- ☐ Gather feedback continuously.
- ☐ Track platform adoption and developer satisfaction.
- ☐ Avoid imposing standards only through top-down mandates.

- ☐ Demonstrate the value of the preferred platform path so teams choose it voluntarily.

Limit Primitives Without Creating Excessive Overhead

- ☐ Decide which cloud and OSS primitives should be officially supported.
- ☐ Remove low-value or duplicate choices.
- ☐ Create opinionated defaults.
- ☐ Allow exceptions when there is a legitimate business or technical reason.
- ☐ Avoid forcing every application team to become experts in infrastructure technologies.
- ☐ Centralize specialized expertise where it can serve many teams.

Reduce Per-Application Glue

- ☐ Identify infrastructure configuration repeated across applications.
- ☐ Move reusable Terraform/IaC logic into shared platform components.
- ☐ Avoid turning a central infrastructure team into a ticket-driven “Terraform writing service.”
- ☐ Build reusable abstractions instead of repeatedly fulfilling individual configuration requests.
- ☐ Give platform engineers enough software-development capability to build durable platform products.
- ☐ Encapsulate underlying systems behind stable interfaces.

Manage Migrations Centrally

- ☐ Reduce the number of underlying technologies that may require future migration.
- ☐ Encapsulate vendor and OSS systems behind APIs where practical.
- ☐ Maintain visibility into which applications depend on each platform component.
- ☐ Collect dependency and usage metadata.
- ☐ Create migration tooling that minimizes work for application teams.
- ☐ Make platform teams responsible for migration automation whenever possible.
- ☐ Plan for technology replacement as a normal lifecycle activity.

Enable “You Build It, You Run It”

- ☐ Ensure application teams can operate the software they develop.
- ☐ Hide unnecessary infrastructure complexity behind platform abstractions.

- ☐ Provide resilient defaults for networking, compute, storage, deployment, and runtime concerns.
- ☐ Minimize operational issues caused by infrastructure rather than application code.
- ☐ Make platform reliability high enough that application teams can trust the abstractions.
- ☐ Keep platform operational responsibility with the platform team for shared infrastructure.

Build the Right Platform Team

- ☐ Combine software engineering and systems/infrastructure expertise.
- ☐ Give the team a mission broader than maintaining infrastructure.
- ☐ For infrastructure engineers, balance operational robustness with developer-focused simplicity.
- ☐ For DevTools engineers, balance developer experience with production-support requirements.
- ☐ For DevOps engineers, replace application-specific automation with reusable capabilities.
- ☐ For SRE engineers, balance reliability with agility, security, performance, and cost.
- ☐ Organize teams around platform outcomes rather than traditional organizational silos.

Support Innovation Without Losing Control

- ☐ Make common paths easy and well supported.
- ☐ Allow teams to experiment when platform capabilities do not meet a legitimate need.
- ☐ Avoid forcing every new idea into the existing platform.
- ☐ Establish a path for successful experiments to become supported platform capabilities.
- ☐ Watch for useful “shadow platforms” that indicate unmet user needs.
- ☐ Avoid building abstractions so generic that they recreate the complexity they were meant to remove.

Success Criteria

- ☐ Application teams ship faster.
- ☐ Developers spend less time managing infrastructure.

- ☐ Operational support requirements decrease.
- ☐ The amount of application-specific integration glue decreases.
- ☐ Fewer infrastructure and OSS primitives need direct application-team expertise.
- ☐ Upgrades and migrations require less effort from the application team.
- ☐ Platform adoption grows because developers find it useful.
- ☐ Reliability remains strong while developer autonomy improves.
- ☐ The organization gains leverage: **a small platform team makes a much larger engineering organization more productive.**